

Relatório Técnico

Generalização da Rede Neural — `exemplo4.py`

Autor: Maurício Rodrigues da Silva

Disciplina: Matemática para Ciência de Dados

Projeto: Generalização do `exemplo4.py`

Autor: Maurício Rodrigues da Silva Disciplina: Matemática para Ciência de Dados

Objetivo

O trabalho consiste em **projetar e implementar uma rede neural multicamadas (MLP)** a partir do arquivo `exemplo4.py` fornecido em aula. Tenho liberdade para modificar a arquitetura, as funções de ativação, o otimizador, o dataset e a estratégia de treino.

Minha proposta foi **generalizar** o código original, antes escrito com índices fixos e ativações *hard-coded*, para uma forma **vetorizada**, capaz de suportar diferentes tamanhos de camada, ativações e datasets, além de incluir uma **verificação numérica do gradiente** para garantir a corretude do backpropagation.

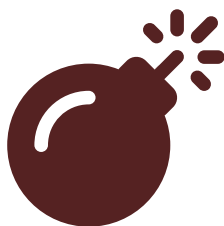
O que foi modificado em relação ao `exemplo4.py`

Item	<code>exemplo4.py</code> (original)	<code>rede_neural_generalizada.py</code> (atual)
Dataset	<code>make_moons</code>	<code>make_circles</code> (mais difícil)
Nº amostras	100	200
Normalização	Não	Sim (z-score)
Neurônios ocultos	2	8
Ativação oculta	<code>sigmoid</code>	<code>tanh</code>
Ativação de saída	<code>sigmoid</code>	<code>sigmoid</code>
Forward/Backward	índices fixos (<code>w0[0,0]</code> , ...)	vetorizado (produto matricial)
Otimizador	GD puro	GD com momento
Treino	batch completo	mini-batch (16 amostras)
Nº épocas	10.000	200
Verificação de gradiente	Não	Sim (diferenças finitas)

Item	exemplo4.py (original)	rede_neural_generalizada.py (atual)
Plots	apenas scatter	scatter + curva de loss + fronteira de decisão

Arquitetura da Rede

MLP totalmente conectada, com forward pass vetorizado: 2 entradas $\rightarrow$ 8 neurônios na camada oculta (`tanh`) $\rightarrow$ 1 saída (`sigmoid`).



Syntax error in text
mermaid version 12.0.0

$$v^{(0)} = W^{(0)}x + b^{(0)}, \quad y^{(0)} = f(v^{(0)})$$

$$v^{(1)} = W^{(1)}y^{(0)} + b^{(1)}, \quad y^{(1)} = g(v^{(1)})$$

onde $x \in \mathbb{R}^d$ (entrada), $W^{(0)} \in \mathbb{R}^{n_h \times d}$, $b^{(0)} \in \mathbb{R}^{n_h}$, $W^{(1)} \in \mathbb{R}^{1 \times n_h}$, $b^{(1)} \in \mathbb{R}$, $f = \text{tanh}$ (camada oculta, $n_h = 8$) e $g = \text{sigmoid}$ (camada de saída).

A perda usada é o erro quadrático médio (MSE) por amostra:

$$L = \frac{1}{2}(y^{(1)} - d)^2$$

Derivação Analítica do Backpropagation

Camada de saída

$$\frac{\partial L}{\partial y^{(1)}} = y^{(1)} - d, \quad \frac{\partial L}{\partial v^{(1)}} = \frac{\partial L}{\partial y^{(1)}} \cdot g'(v^{(1)})$$

Para a sigmoide, $g'(v) = y(1 - y)$, logo:

$$\frac{\partial L}{\partial v^{(1)}} = (y^{(1)} - d) y^{(1)} (1 - y^{(1)})$$

$$\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial v^{(1)}} \cdot (y^{(0)})^T, \quad \frac{\partial L}{\partial b^{(1)}} = \frac{\partial L}{\partial v^{(1)}}$$

Camada oculta

Propagando o erro para trás (regra da cadeia):

$$\frac{\partial L}{\partial y^{(0)}} = (W^{(1)})^T \frac{\partial L}{\partial v^{(1)}}, \quad \frac{\partial L}{\partial v^{(0)}} = \frac{\partial L}{\partial y^{(0)}} \odot f'(v^{(0)})$$

($\odot$ = produto de Hadamard). Para `tanh`, $f'(v) = 1 - y^2$:

$$\frac{\partial L}{\partial v^{(0)}} = \frac{\partial L}{\partial y^{(0)}} \odot (1 - (y^{(0)})^2)$$

$$\frac{\partial L}{\partial W^{(0)}} = \frac{\partial L}{\partial v^{(0)}} \cdot x^T, \quad \frac{\partial L}{\partial b^{(0)}} = \frac{\partial L}{\partial v^{(0)}}$$

Tabela de ativações e derivadas

Ativação	$f(v)$	$f'(v)$
Sigmoide	$\frac{1}{1 + e^{-v}}$	$y(1 - y)$
Tanh	$\tanh(v)$	$1 - y^2$
ReLU	$\max(0, v)$	1 se $v > 0$, senão 0
Leaky ReLU	$\max(0.01 v, v)$	1 se $v > 0$, senão 0.01

No código, cada ativação é implementada como um par de funções (`sigmoid / sigmoid_prime`, `tanh / tanh_prime`, `relu / relu_prime`), e a derivada é calculada a partir da **saída** y (mais eficiente e estável), exceto para a ReLU, cuja derivada depende da entrada v .

Observação sobre Cross-Entropy

Se a perda MSE for substituída por *binary cross-entropy* mantendo a saída sigmoide:

$$L = -[d \log y^{(1)} + (1 - d) \log(1 - y^{(1)})] \implies \frac{\partial L}{\partial v^{(1)}} = y^{(1)} - d$$

o termo $y(1 - y)$ cancela com o denominador da cross-entropy, tornando o gradiente mais estável numericamente. Essa simplificação não foi adotada no projeto (optamos por manter o MSE, como no `exemplo4.py` original), mas fica registrada como possível evolução do trabalho.

Verificação Numérica do Gradiente

Para validar a implementação do backward, cada gradiente analítico é comparado com uma aproximação por diferenças finitas centradas:

$$\frac{\partial L}{\partial \theta} \approx \frac{L(\theta + \varepsilon) - L(\theta - \varepsilon)}{2\varepsilon}, \quad \varepsilon = 10^{-5}$$

Isso é feito percorrendo elemento a elemento de cada parâmetro ($W^{(0)}$, $b^{(0)}$, $W^{(1)}$, $b^{(1)}$), perturbando-o em $\pm \varepsilon$ e recalculando a perda (função `numerical_grad`). O erro relativo máximo obtido na execução (função `check_gradients`), para uma amostra do dataset, foi:

Parâmetro	Erro relativo máx.
<code>w0</code>	1.77e-10
<code>b0</code>	2.56e-10
<code>w1</code>	2.48e-10
<code>b1</code>	5.02e-12

Todos os erros estão muito abaixo do limiar de 10^{-6} , confirmando que o backpropagation implementado está correto.

Os valores exatos podem variar levemente a cada execução, pois a geração do dataset (`make_circles`) não usa semente fixa; apenas a inicialização dos pesos e o treino são reproduzíveis (`np.random.seed`).

Resultado do Treinamento

- Dataset: `make_circles` (200 amostras, normalizado), não linearmente separável.
- Otimizador: gradiente descendente com momento (`momentum = 0.9`), mini-batches de 16 amostras, 200 épocas, `lr = 0.1`.
- A loss média cai de ≈ 0.125 (época 0) para ≈ 0.012 (época 180).
- **Acurácia final:** $\approx 97\text{-}99\%$ (varia levemente entre execuções, pois o dataset não usa semente fixa).

O script gera as três figuras abaixo.

Dataset normalizado

 Dataset make_circles normalizado

Os dois círculos concêntricos (classes azul e vermelha) mostram um problema **não linearmente separável**: não existe uma reta capaz de separar as duas classes, o que justifica o uso de uma camada oculta com ativação não linear (`tanh`).

Curva de aprendizado

 Curva de loss por época

A loss média (MSE) cai de forma consistente ao longo das 200 épocas, sem oscilações bruscas, evidência de que o gradiente com momento e o treino em mini-batch estão convergindo de forma estável. A curva achata-se após ≈ 150 épocas, indicando que a rede já se aproxima de um mínimo local satisfatório para o problema.

Fronteira de decisão

 Fronteira de decisão aprendida

A região colorida mostra como a rede classifica cada ponto do plano. A fronteira aprendida acompanha o formato circular esperado, separando o círculo interno do externo, confirmando que os 8 neurônios da camada oculta com `tanh` foram suficientes para capturar a não linearidade do problema. Os poucos pontos classificados incorretamente ficam concentrados na fronteira entre as duas classes, onde o ruído do dataset (`noise=0.1`) mais se sobrepõe.

Arquivos do repositório

Arquivo	Descrição
<code>exemplo4.py</code>	Código original fornecido em aula (rede fixa em 2 neurônios, índices <i>hard-coded</i> , dataset <code>make_moons</code>).
<code>rede_neural_generalizada.py</code>	Minha implementação: rede vetorizada e genérica, com verificação numérica de gradiente, treino em mini-batch com momento e geração dos gráficos de resultado. Renomeado a partir de <code>atividade.py</code> .

Arquivo	Descrição
requirements.txt	Dependências do projeto (numpy , matplotlib , scikit-learn).

Como executar

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python rede_neural_generalizada.py
```